

EXAMEN FINAL — BASE DE DONNÉES AVANCÉE

SIEM

Security Information & Event Management

Conception et programmation avancée d'une base de données de centralisation des alertes de cybersécurité.

PostgreSQL

MongoDB

Python

Streamlit

Plan de la présentation

01

Contexte & problématique

Pourquoi un SIEM ? quel besoin réel ?

02

Architecture du système

PostgreSQL · MongoDB · ETL Python · Dashboard

03

Partie I — PostgreSQL

Tables, séquences, fonctions, triggers, boucles, vue

04

Partie II — MongoDB

Collection 19 champs, requêtes & agrégations

05

Pipeline complet & dashboard

Démonstration en direct

06

Conclusion & perspectives

Synthèse · difficultés · améliorations

Le problème : trop d'alertes, partout

Une entreprise moyenne reçoit 10 000+ alertes par jour, venant de sources hétérogènes.

Firewalls

tentatives bloquées, scans

Serveurs

logs d'authentification

Postes

antivirus, anomalies

Bases données

requêtes suspectes

→ Un analyste ne peut pas tout regarder. Il faut centraliser, prioriser, automatiser.

VOLUME

350+

alertes en 7 jours

PRIORISATION

0 → 100

score automatique par trigger

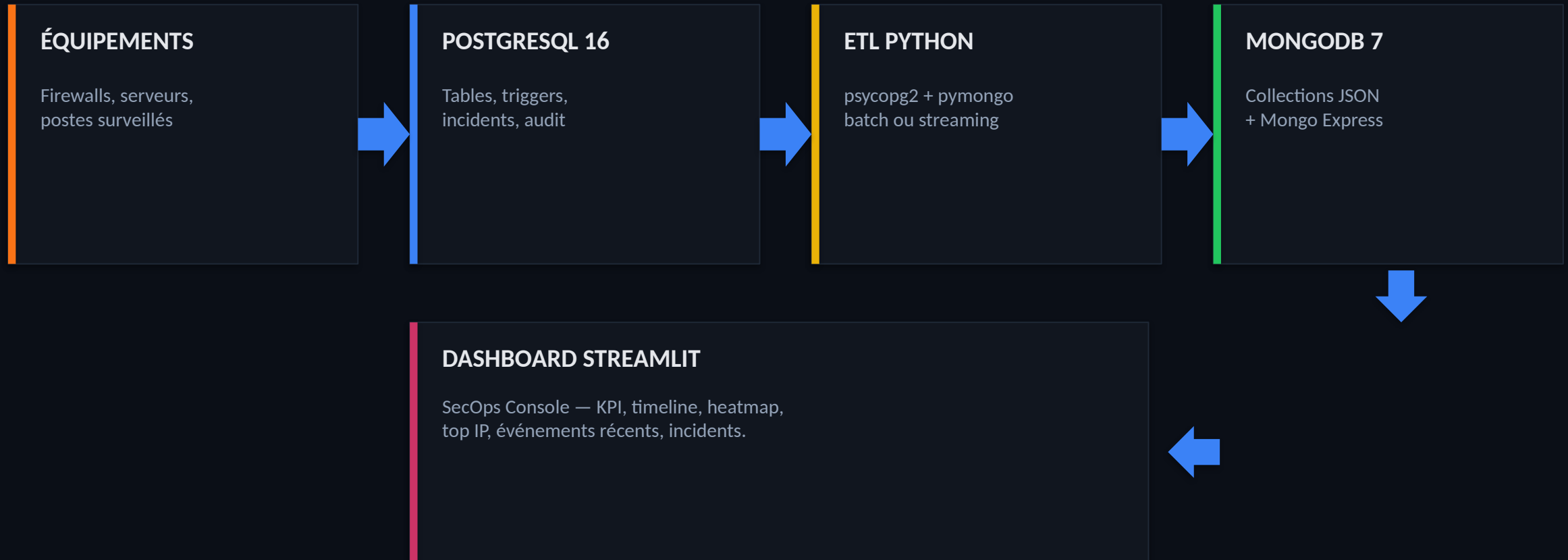
TEMPS DE RÉACTION

< 1 s

via streaming LISTEN/NOTIFY

Architecture du système

PostgreSQL = source de vérité (ACID). MongoDB = couche d'affichage rapide.



PostgreSQL — 6 tables liées

CDC : minimum 3 tables avec PK / FK. Réalisé : 6 tables.

TABLE	RÔLE MÉTIER	LIGNES
équipement	Machines surveillées (INET, type SERVEUR/FIREWALL/...)	8
analyste	Équipe SOC. email_hash = SHA-256 (RGPD via pgcrypto)	4
regle_detection	Signatures (BRUTE_FORCE, DDOS, MALWARE...)	8
alerte	Événements détectés. payload JSONB indexé GIN.	350+
incident	Agrégation auto d'alertes critiques. CHECK dates.	2+
journal_audit	Hash chaîné SHA-256, audit infalsifiable.	auto

Types avancés : INET (IP natif), JSONB (payload brut), NUMERIC (score).

Logique métier dans la base

Tout est déclaré et enforced côté PostgreSQL.

6 SÉQUENCES

CDC : ≥ 2

```
seq_equipement, seq_analyste,  
seq_regle, seq_alerte,  
seq_incident, seq_audit
```

6 FONCTIONS PL/pgSQL

CDC : ≥ 2

```
calculer_score_risque,  
niveau_risque, hash_audit,  
verifier_integrite_audit, ...
```

4 TRIGGERS

CDC : ≥ 2

```
BEFORE INSERT (score auto),  
AFTER INSERT (incident auto,  
pg_notify), AUDIT (hash)
```

5 PROCÉDURES

CDC : ≥ 2

```
Boucles LOOP, FOR, WHILE  
+ EXCEPTION + curseurs.  
Archivage, rapport, ...
```

4 VUES + 1 MAT.

CDC : 1 vue globale

```
v_alertes_enrichies (4 jointures),  
v_dashboard_soc, v_top_ip,  
mv_dashboard_soc
```

16 INDEX

Performance

```
Composite, partiel,  
GIN sur JSONB,  
trigram sur nom
```

Trigger phare : score auto + incident auto

Toute la logique tient dans un BEFORE INSERT et un AFTER INSERT.

ENTRÉE — INSERT brut (analyste)

```
INSERT INTO alerte (  
    id_equipement, id_regle,  
    source_ip, destination_ip  
)  
VALUES (  
    1, 1,  
    '45.33.18.7',  
    '192.168.10.10'  
);  
  
→ score_risque : NULL  
→ statut : 'NOUVELLE'  
→ payload : NULL
```



APRÈS — Triggers exécutés

```
score_risque = 90 (BEFORE)  
              calculé selon :  
                - sévérité règle  
                - heure (nuit +10)  
  
+ pg_notify('alertes_channel'...)  
  → script Python streaming reçoit  
    en < 1 s  
  
AFTER : si 5+ critiques en 1 h  
        → INSERT INTO incident  
          (titre auto-généré)  
  
+ trigger AUDIT : ligne signée  
  SHA-256 ajoutée à journal_audit
```

MongoDB — collection riche en sous-documents

CDC : ≥ 15 champs par document. Réalisé : 19 champs.

alertes — exemple de document

```
{
  "_id"           : ObjectId("..."),
  "id_alerte"     : 287,
  "date_alerte"   : ISODate("2026-05-03T14:22"),
  "score_risque"  : 87.0,
  "niveau_calcule": "CRITIQUE",
  "statut"        : "NOUVELLE",
  "source"        : { "ip": "45.33.18.7",
                      "port": 22 },
  "destination"   : { "ip": "192.168.10.10" },
  "equipement"    : { "id": 1, "nom": "SRV-WEB-01",
                      "type": "SERVEUR",
                      "ip": "192.168.10.10",
                      "localisation": "Datacenter-Casa" },
  "regle"         : { "id": 1, "code": "BF-001",
                      "categorie": "BRUTE_FORCE",
                      "severite": "HIGH",
                      "description": "SSH brute force" },
  "payload"       : { "user": "root",
                      "tentatives": 52 }
}
```

TOTAL CHAMPS

19+

champs interrogeables

DOCUMENTS

350+

alertes en MongoDB

INDEX

5

(date, IP, sévérité, équip., url)

MongoDB — 7 types de requêtes (CDC)

Tout est dans 08_mongodb_requetes.js et démontrable en live.

Condition numérique

```
db.alertes.find({ score_risque: { $gt: 70 } })
```

Recherche textuelle

```
db.alertes.find({ "source.ip": { $regex: "^45\\.\\. " } })
```

Suppression

```
db.alertes.deleteMany({ "regle.severite": "LOW" })
```

Projection

```
db.alertes.find({}, { id_alerte:1, score_risque:1,  
  "source.ip":1, _id:0 }).sort({ score_risque:-1 })
```

Multi-conditions AND/OR

```
db.alertes.find({ $or: [ { "regle.severite":"CRITICAL" },  
  { score_risque: { $gt: 80 } } ] })
```

Mise à jour

```
db.alertes.updateMany( { "source.ip":"203.0.113.99" },  
  { $set: { statut: "FAUX_POSITIF" } } )
```

Agrégation \$avg/\$sum/\$max

```
db.alertes.aggregate([ { $group: { _id: "$regle.categorie",  
  avg: { $avg: "$score_risque" }, total:{ $sum: 1 } } } ])
```

Le pipeline complet en 4 étapes

De l'événement brut au tableau de bord.

1

INSERTION

INSERT dans alerte (analyste / sonde)

Trigger BEFORE → score auto

2

DÉTECTION

Trigger AFTER vérifie les seuils

5+ alertes critiques / h → INCIDENT auto

3

DIFFUSION

pg_notify('alertes_channel') ou ETL batch

Script Python lit puis écrit dans Mongo

4

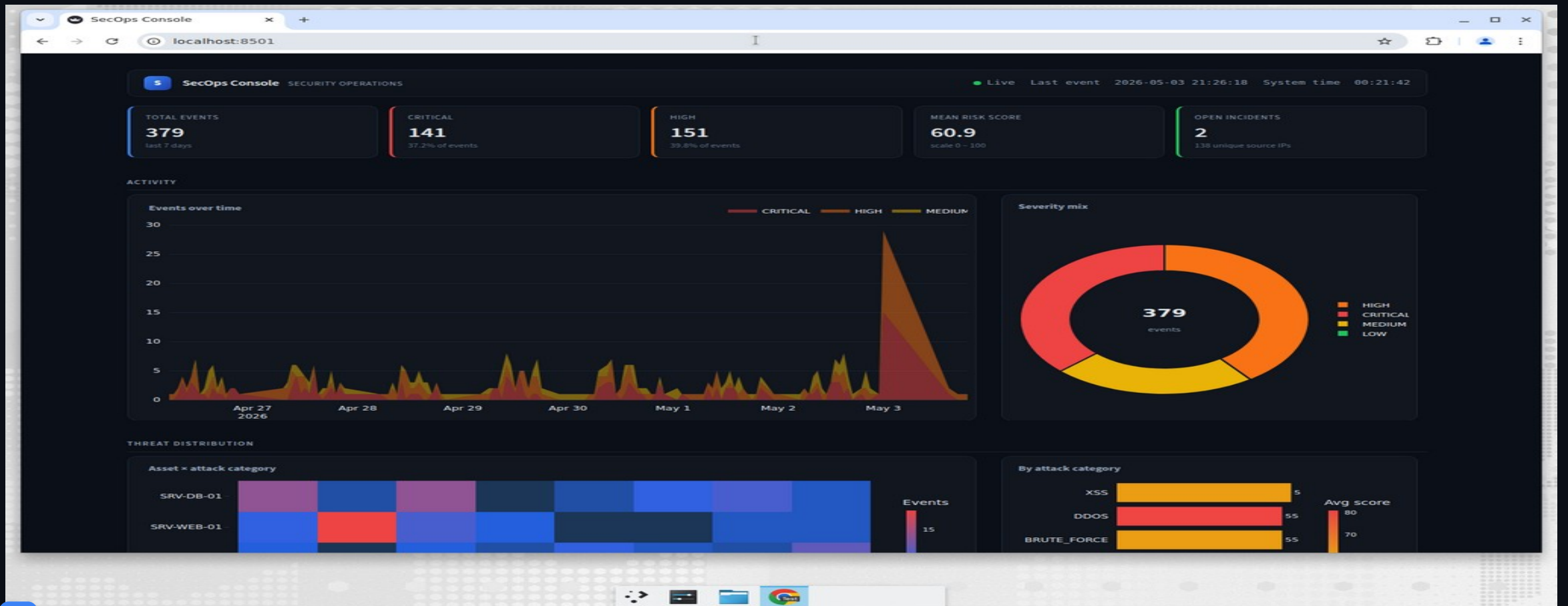
AFFICHAGE

Dashboard Streamlit (SecOps Console)

KPI, timeline, top IP, heatmap, incidents

Le dashboard SecOps Console

Streamlit + Plotly, lisant les collections MongoDB en temps réel.



Conformité au cahier des charges (20 / 20)

PROJET A — PostgreSQL

12 / 12 points

✓ 3+ tables PK/FK	6 tables	2/2
✓ ≥ 2 séquences	6 séquences	2/2
✓ ≥ 2 fonctions PL/pgSQL	6 fonctions	2/2
✓ ≥ 2 triggers	4 triggers	2/2
✓ ≥ 2 boucles	5 procédures	2/2
✓ Vue globale	5 vues + matérialisée	2/2

PROJET B — MongoDB

8 / 8 points

✓ Collection ≥ 15 champs	19 champs	2/2
✓ Condition numérique	\$gt, \$gte, \$lt	
✓ Conditions multiples	\$and / \$or / \$ne	
✓ Recherche textuelle	\$regex sur sous-doc	
✓ Update / Delete	updateMany / deleteMany	
✓ Agrégations	\$group + \$facet + \$lookup	
✓ Projection	find({}, {...})	6/6

Difficultés rencontrées & perspectives

DIFFICULTÉS RENCONTRÉES

- Choix entre normalisation PG et enrichissement Mongo (sous-docs)
- Ordre des triggers BEFORE/AFTER pour éviter les races
- Sérialisation INET/NUMERIC depuis PG vers Mongo (typage)
- Setup Docker sous Windows 11 (WSL2, ports, healthcheck)

PERSPECTIVES D'AMÉLIORATION

- Streaming temps réel (LISTEN/NOTIFY déjà en bonus)
- Détection ML : IsolationForest sur scores anormaux
- Géolocalisation IP attaquantes (geoip2 + folium)
- API REST FastAPI + ticketing (intégration SOAR)
- Row Level Security (analyste JUNIOR ne voit que ses incidents)

MERCI POUR VOTRE ATTENTION

Questions ?

Toute la base, le code, le rapport et les diagrammes
sont disponibles dans le dossier `projet_siem/`.

YASSIR ZHD

CI1 G1 — Génie Informatique • Spé. Cybersécurité

Pr. Achraf Chakir BARAKA

S

SIEM